**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Дисциплина:** Фронт-энд разработка

Отчет

# Лабораторная работа №2

Выполнил:

Смирнова Карина

К3441

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

# Задача

Варианты остаются прежними. Теперь Вам нужно привязать то, что Вы делали в ЛР1 к внешнему API средствами fetch/axios/xhr. Реализуйте моковое API средствами JSON-сервера и подключите к нему авторизацию, как в примерах, которые мы рассматривали в рамках тем "Иммитация работы с API".

Например, для приложения для просмотра прогнозов погоды задание выглядит следующим образом:

Реализовать получение погоды (прогноз на ближайшие 7 дней) из открытого API OpenWeatherMap, взависимости от геолокации пользователя. Реализовать вывод полученного прогноза в виде 7 карточек в три ряда (первый ряд - крупная карточка, второй ряд - три карточки в меньшем размере, третий ряд - четыре карточки в маленьком размере).

# Ход работы

Изначально был составлен список endpoint для покрытия функционала сверстаных страниц.

| Страница | Запрос | Назначение |
|---|---|---|
| Вход | POST /login | Проверка почты и пароля, получение токена |
| Регистрация | POST /register | Создание пользователя |
| Поиск | GET /listings | Получение списка объявлений с фильтрацией и сортировкой |
| Страница объекта | GET /listings/id | Получение одного объявления по его идентификатору |
| Страница объекта | POST /bookings | Создание заявки на бронирование |
| Профиль | PATCH /users/id | Изменение имени, почты и телефона пользователя |
| Профиль | GET /listings?userId=id | Получение объявлений пользователя |
| Профиль | PATCH /listings/id | Скрытие или повторная публикация объявления |
| Профиль и активность | GET /bookings | Получение бронирований и сделок |
| Активность | PATCH /bookings/id | Изменение статуса бронирования, например его отмена |

| | | |
|---|---|---|
| Активность | GET /chats?userId=id | Получение списка чатов пользователя |
| Активность | PATCH /chats/id | Обновление последнего сообщения в чате |
| Активность | GET /messages?chatId=id | Получение сообщений выбранного чата |
| Активность | POST /messages | Отправка нового сообщения |

После определения запросов в папке лабораторной работы был инициализирован npm-проект. Для этого был создан файл package.json. Затем были установлены зависимости json-server, json-server-auth и axios. JSON Server использовался для создания мокового API, а json-server-auth - для регистрации, входа и проверки токена пользователя. Библиотека Axios использовалась для отправки запросов из JavaScript. Также в package.json была добавлена команда запуска сервера на порте 3000.

Следующим этапом был создан файл db.json, который выполняет роль базы данных. Сначала в него были добавлены основные коллекции: users, listings и bookings. В коллекции users хранятся пользователи, в listings - объявления об аренде, а в bookings - заявки на бронирование.

Для настройки прав доступа был создан файл routes.json. Просматривать объявления можно без авторизации, но изменять их может авторизованный пользователь. Работа с пользователями, бронированиями, чатами и сообщениями выполняется с использованием токена. Токен передается в заголовке запроса Authorization.

После подготовки базы был реализован файл api.js. В нём был создан общий экземпляр Axios с адресом http://localhost:3000. Затем были написаны отдельные асинхронные функции для регистрации, входа, загрузки объявлений, получения одного объекта и создания бронирования.

После этого была переработана логика HTML-страниц и файла app.js. Для форм входа и регистрации были добавлены отдельные обработчики. Вход выполняется только после успешного ответа сервера.

Полученный токен и информация о пользователе сохраняются в localStorage.

На странице поиска статические карточки были заменены загрузкой объявлений из db.json. После открытия страницы выполняется запрос GET /listings, а полученные данные используются для создания карточек. В запрос также передаются параметры фильтрации по типу объекта, городу, стоимости и формату аренды. Идентификатор объявления читается из адресной строки, например listing.html?id=1. После этого отправляется запрос GET /listings/1, и данные объекта подставляются в страницу. При выборе дат рассчитывается итоговая стоимость. Если пользователь авторизован, он может отправить заявку с помощью запроса POST /bookings.

После реализации основной части результат был протестирован. После проверки было решено добавить работу с чатами и сообщениями. Для этого в db.json появились коллекции chats и messages. Список чатов загружается запросом GET /chats, а история выбранного чата — запросом GET /messages. При отправке сообщения создается новая запись через POST /messages.

Несколько примеров функционала:

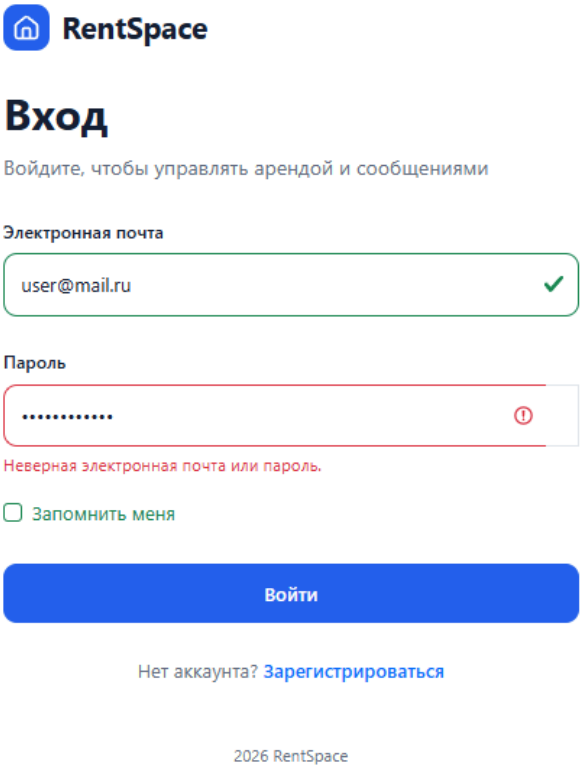



Рисунок 1 - авторизация с некорректными данными

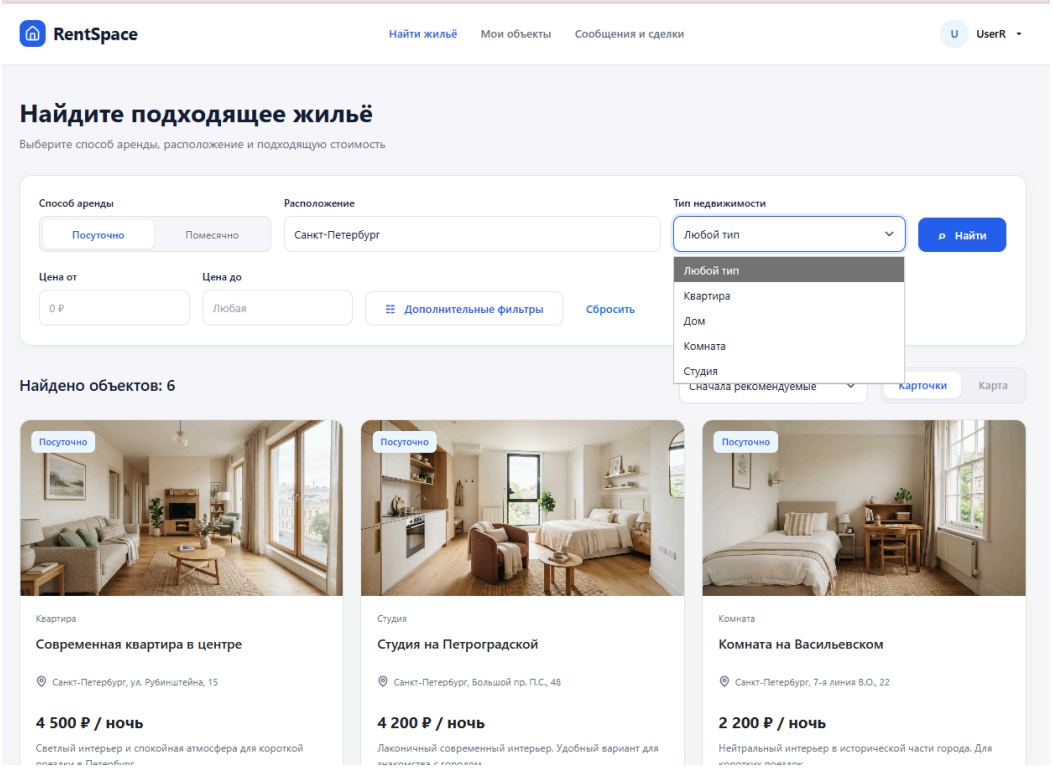



Рисунок 2 - страница поиска объявлений

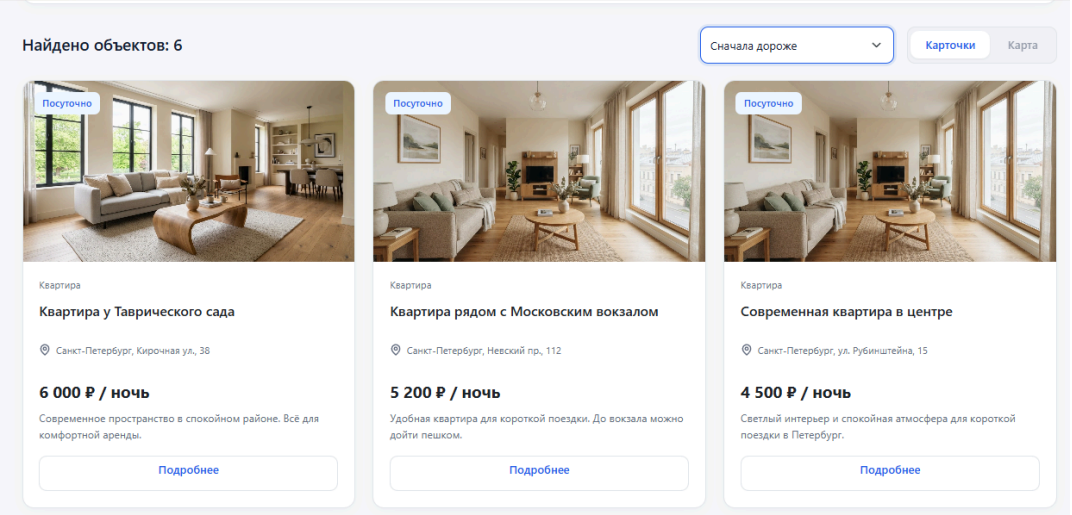



Рисунок 3 - фильтрация по цене

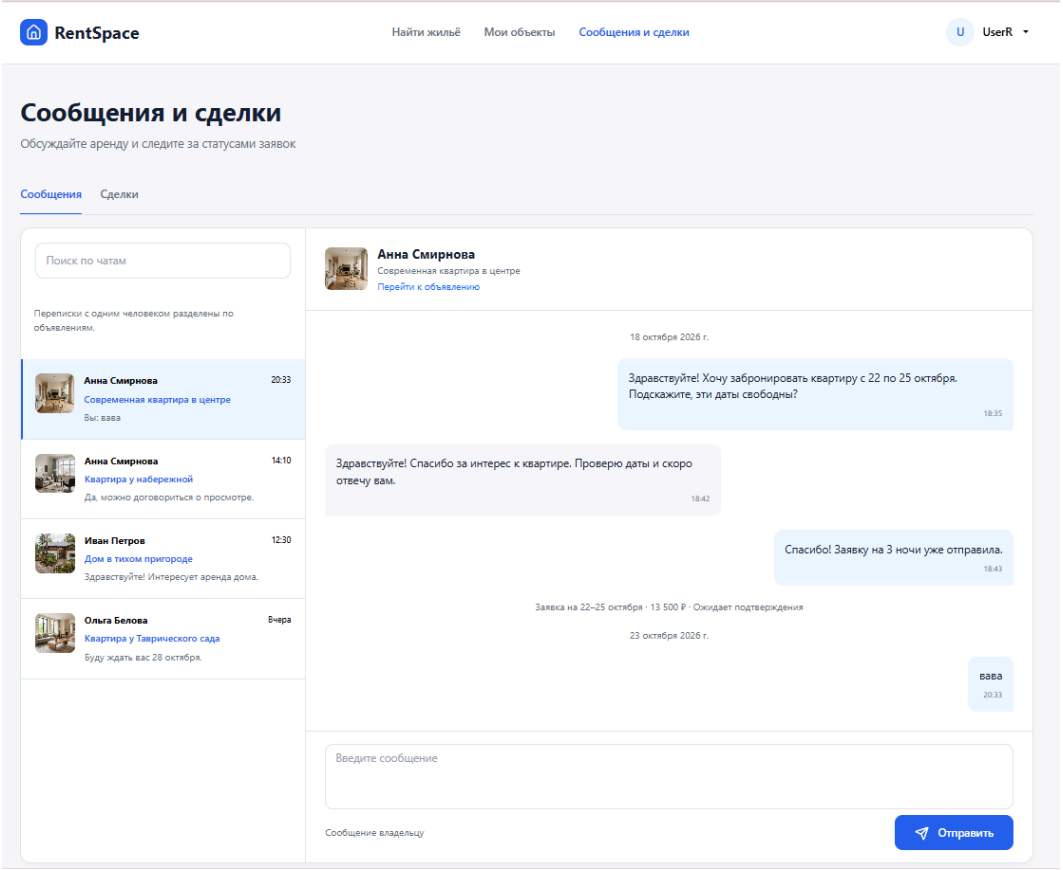



Рисунок 3 - страница чатов

# Вывод

В ходе лабораторной работы был создан моковый API с помощью JSON Server. Я научилась отправлять запросы с использованием Axios, получать и обрабатывать ответы сервера, а также выводить полученные данные на HTML-страницы. Кроме того, я научилась добавлять, изменять и загружать данные из db.json, работать с авторизацией и обрабатывать возможные ошибки при выполнении запросов.